Task 1: In this task, you will compare the recursion with iteration approach for doing repeated execution of tasks.

1. Compare iterative vs. recursive implementation for three test cases above for different data sizes.

SUM OF NATURAL NUMBERS:
- Both approaches produce identical results.
- Iterative is faster because recursion has overhead from creating a new stack frame for every function call.

FIBONACCI SEQUENCE:
- Recursive Fibonacci is EXTREMELY slow for larger n because it recalculates the same values over and over (exponential time complexity: O(2^n)).
- Iterative Fibonacci runs in linear time O(n), making it dramatically faster even at modest sizes like n=30.

FACTORIAL:
- Both approaches produce identical results.
- Iterative is generally faster due to avoiding the overhead of recursive function calls and stack frame allocation.

GENERAL CONCLUSION:
- Iterative solutions are typically faster in Python because each recursive call adds overhead (stack frame creation, function call setup).
- Recursion can hit Python's recursion limit for large inputs.
- Recursion is often more elegant and easier to read, but iteration is more efficient for performance-critical code.

Task 2:

1. Test the merge-sort algorithm for different data sizes

```
The sum of the first 5 natural numbers is 15
The 7th Fibonacci number is 13
The factorial of 5 is 120

Data size: 100
Insertion Sort took: 0.001815 seconds
Merge Sort took: 0.001327 seconds

Data size: 500
Insertion Sort took: 0.042013 seconds
Merge Sort took: 0.007312 seconds

Data size: 1000
Insertion Sort took: 0.169593 seconds
Merge Sort took: 0.017461 seconds

Data size: 5000
Insertion Sort took: 4.170690 seconds
Merge Sort took: 0.095089 seconds

Data size: 10000
Insertion Sort took: 17.685744 seconds
Merge Sort took: 0.200392 seconds
[2, 6, 9, 5, 3, 4, 5, 6, 6, 7]

[2, 3, 4, 5, 5, 6, 6, 6, 7, 9]
[1]
[1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 3, 3, 3, 3, 3, 3, 3, 3, 4, 4, 4, 4, 4, 4, 4, 4, 4, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 6, 6, 6]
3240
```

2. Test an inefficient sort, such as insertion sort algorithm with similar test data sequences as in (1)

```
Data size: 100
Insertion Sort took: 0.001514 seconds
Merge Sort took: 0.001220 seconds

Data size: 500
Insertion Sort took: 0.036729 seconds
Merge Sort took: 0.007265 seconds

Data size: 1000
Insertion Sort took: 0.166465 seconds
Merge Sort took: 0.016566 seconds

Data size: 5000
Insertion Sort took: 4.142998 seconds
Merge Sort took: 0.094979 seconds

Data size: 10000
Insertion Sort took: 17.027822 seconds
Merge Sort took: 0.201029 seconds
[2, 6, 9, 5, 3, 4, 5, 6, 6, 7]

[2, 3, 4, 5, 5, 6, 6, 6, 7, 9]

Insertion Sort test cases:
Data size: 10 - First 10 sorted: [22, 160, 172, 334, 450, 507, 582, 706, 768, 864]
Data size: 100 - First 10 sorted: [12, 14, 19, 22, 51, 79, 93, 97, 97, 105]
Data size: 1000 - First 10 sorted: [1, 1, 3, 3, 3, 4, 4, 5, 7, 7]
Data size: 5000 - First 10 sorted: [1, 1, 1, 2, 2, 2, 2, 3, 3, 3]
Data size: 10000 - First 10 sorted: [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
[1]
[1, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 5, 5, 5]
3240
```

3. Benchmark the algorithm efficiency (comparison of merge-sort with insertion sort) with TimeIt module

```
The sum of the first 5 natural numbers is 15
The 7th Fibonacci number is 13
The factorial of 5 is 120

Data size: 100
Insertion Sort took: 0.001815 seconds
Merge Sort took: 0.001327 seconds

Data size: 500
Insertion Sort took: 0.042013 seconds
Merge Sort took: 0.007312 seconds

Data size: 1000
Insertion Sort took: 0.169593 seconds
Merge Sort took: 0.017461 seconds

Data size: 5000
Insertion Sort took: 4.170690 seconds
Merge Sort took: 0.095089 seconds

Data size: 10000
Insertion Sort took: 17.685744 seconds
Merge Sort took: 0.200392 seconds
[2, 6, 9, 5, 3, 4, 5, 6, 6, 7]

[2, 3, 4, 5, 5, 6, 6, 6, 7, 9]
[1]
[1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 3, 3, 3, 3, 3, 3, 3, 3, 4, 4, 4, 4, 4, 4, 4, 4, 4, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 6, 6, 6]
3240
```

Task 3:

Test the following python script for tower of hanoi problem for different disk sizes and report the results for at least 2 different disk sizes.

1.
```
Enter number of disks: 2
The moves are:
Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
```

2.
```
Enter number of disks: 3
The moves are:
Move disk 1 from A to B
Move disk 2 from A to C
Move disk 1 from B to C
Move disk 3 from A to B
Move disk 1 from C to A
Move disk 2 from C to B
Move disk 1 from A to B
```

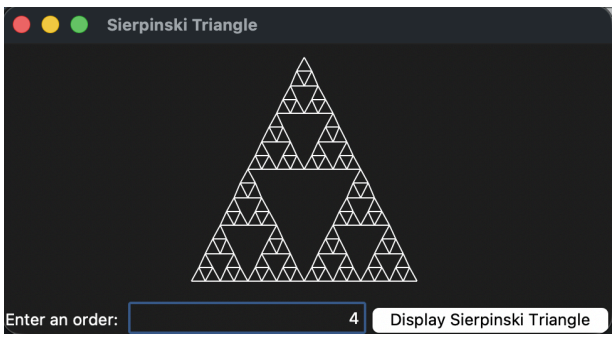
Modify the code for TowerOfHanoi.py, so that the program finds the number of moves needed to move n disks from tower A to tower B. (Hint: Use a global variable and increment it for every move.). Test the code for two different disk sizes and report the results. A successful test run should display the following at the console.

```
Enter number of disks: 8
The moves are:
Number of moves is 255
```

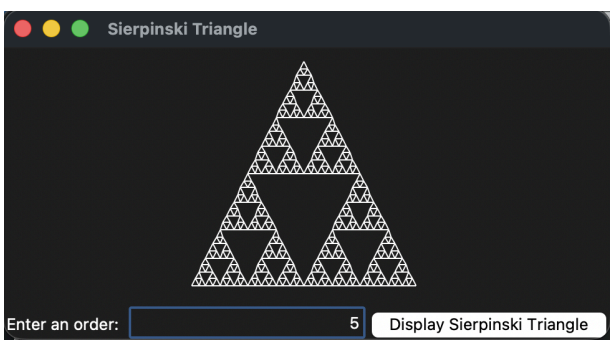
Task 4:

You need to run the TkInter code sierpinski_triangle.py for this simple fractal problem and test it for at least 3 different orders other than 0,1,2,and 3, and report the results(screen shots).

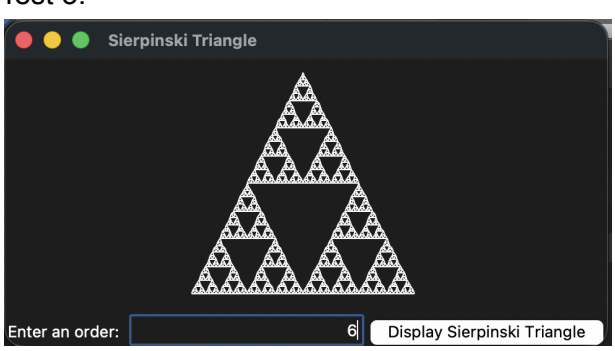
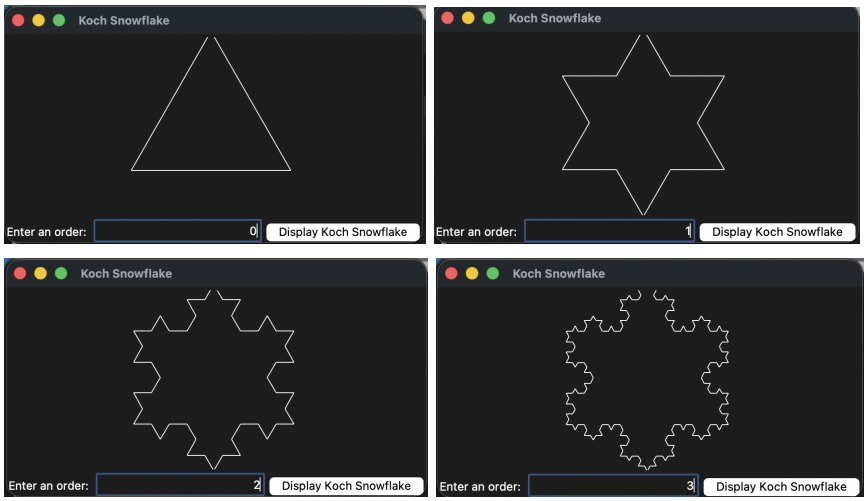
Test 1:



Test 2:



Test 3:



a. Begin with an equilateral triangle, which is considered to be the Koch fractal of order (or level) 0, as shown in Figure a.
b. Divide each line in the shape into three equal line segments and draw an out-ward equilateral triangle with the middle line segment as the base to create a Koch fractal of order 1, as shown in Figure b.
c. Repeat Step 2 to create a Koch fractal of order 2, 3, ..., and so on, as shown in Figure c–d